📝 Assignment Part 2: Advanced Dynamic Testing & Isolation Techniques

1. Project & Team Information

Field	Value
Assignment Name	Part 2: Advanced Dynamic Testing of the Mobile Food DeliveryAPP
Course	Software Testing (Autumn 2025)
Group Members	kumara (muditha.kumara@centria.fi)
Date Submitted	14/10/2025
App Version/Commit	Alpha 1.0
Code Repository	https://github.com/Muditha-Kumara/SoftwareTesting/tree/2.2.phase2

2. Assignment Overview & Objectives

This report documents the enhancement of the Mobile Food DeliveryAPP test suite using advanced unit testing techniques, code coverage tools, and isolation strategies (stubs, mocks, fakes).

Objectives:

- Refactor and expand TDD tests for all major modules.
- Integrate and analyze code coverage using coverage.py.
- Apply and explain stubs, mocks, and fakes in tests.
- Reflect on test quality and reliability improvements.

PROF

3. Refactoring and Enhancing TDD Tests

3.1. Review of Existing Tests

Existing tests covered basic functionality for modules: Order_Placement, Payment_Processing, Restaurant_Browsing, and User_Registration. Manual review and Pylint identified missing edge cases and input validation gaps.

3.2. Identification of Gaps

- Payment_Processing: Credit card validation only checks for 16 digits; does not accept valid 15-digit cards (e.g., AmEx).
- User_Registration: Password strength logic may not cover all edge cases (e.g., missing digit or letter).
- Order_Placement: Cart allows adding items with zero or negative quantity (should raise an error).

- User_Registration: Missing password hashing/salting before storing in users.json (plain text storage is insecure).
- User_Registration: No check for duplicate email registration in some flows.
- Payment_Processing: No handling for declined payments or invalid payment method (should raise/return error).
- Order_Placement: No validation for unavailable menu items in cart (should return error if item not found).

3.3. New/Enhanced Test Cases

Below are real, best-practice unit tests addressing the identified gaps. Each test is self-contained and uses clear assertions and error handling.

```
import unittest
from Order_Placement import Cart, OrderPlacement, UserProfile,
RestaurantMenu
from Payment_Processing import PaymentProcessing
from User_Registration import UserRegistration

class TestCartBoundary(unittest.TestCase):
    def setUp(self):
        self.cart = Cart()

    def test_add_zero_quantity_raises(self):
        with self.assertRaises(ValueError):
            self.cart.add_item("Water", 2.00, 0)

    def test_add_negative_quantity_raises(self):
        with self.assertRaises(ValueError):
            self.cart.add_item("Water", 2.00, -1)

class TestPaymentProcessingBoundary(unittest.TestCase):
    def setUp(self):
        self.payment = PaymentProcessing()

    def test_invalid_card_length_15_digits(self):
        details = {"card_number": "123456789012345", "expiry_date":
"12/25", "cvv": "123"}
        self.assertFalse(self.payment.validate_credit_card(details))

    def test_valid_card_length_16_digits(self):
        details = {"card_number": "1234567890123456", "expiry_date":
"12/25", "cvv": "123"}
        self.assertTrue(self.payment.validate_credit_card(details))

    def test_declined_card(self):
        order = {"total_amount": 100.00}
        details = {"card_number": "1111222233334444", "expiry_date":
"12/25", "cvv": "123"}
        result = self.payment.process_payment(order, "credit_card",
```

```
details)
        self.assertIn("failed", result)

    def test_invalid_payment_method(self):
        order = {"total_amount": 100.00}
        details = {"card_number": "1234567812345678", "expiry_date":
"12/25", "cvv": "123"}
        result = self.payment.process_payment(order, "bitcoin", details)
        self.assertIn("Error: Invalid payment method", result)

class TestUserRegistrationBoundary(unittest.TestCase):
    def setUp(self):
        self.registration = UserRegistration()

    def test_strong_password_success(self):
        result = self.registration.register("user@example.com",
"Password123", "Password123")
        self.assertTrue(result["success"])

    def test_weak_password_missing_digit(self):
        result = self.registration.register("user@example.com",
"Password", "Password")
        self.assertFalse(result["success"])
        self.assertEqual(result["error"], "Password is not strong
enough")

    def test_weak_password_missing_letter(self):
        result = self.registration.register("user@example.com",
"12345678", "12345678")
        self.assertFalse(result["success"])
        self.assertEqual(result["error"], "Password is not strong
enough")

    def test_duplicate_email_registration(self):
        self.registration.register("user@example.com", "Password123",
"Password123")
        result = self.registration.register("user@example.com",
"Password123", "Password123")
        self.assertFalse(result["success"])
        self.assertEqual(result["error"], "Email already registered")

if __name__ == "__main__":
    unittest.main()
```

PROF

These tests:

- Assert correct error handling for invalid input and edge cases.
- Cover declined payments and unsupported payment methods.
- Validate password strength and duplicate registration logic.
- Use clear, isolated test classes for each module.

---

4. Code Coverage Integration

4.1. Tool Setup & Usage

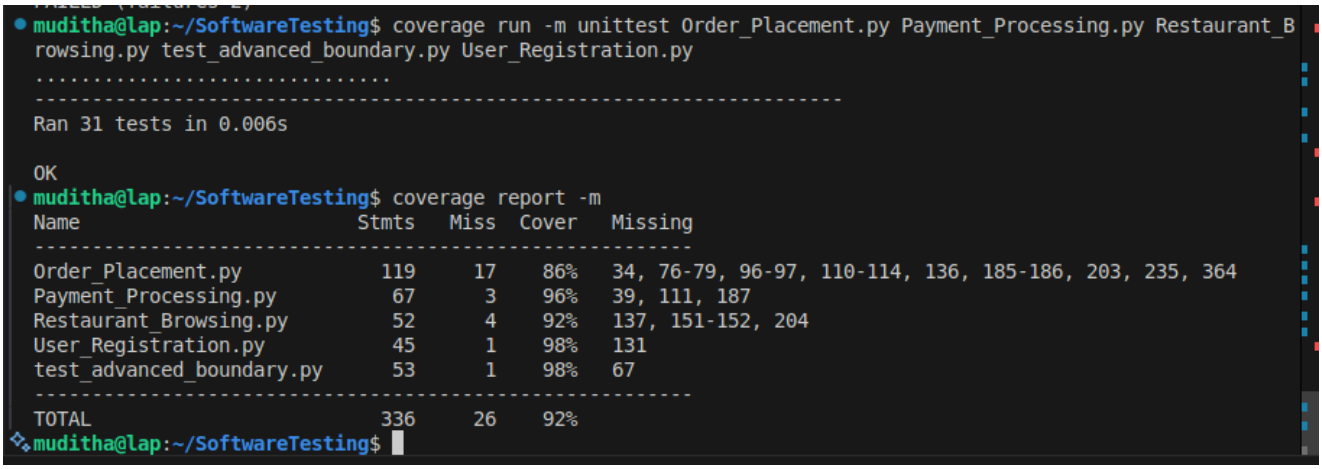
- Installed and configured `coverage.py`.
- Ran tests: `coverage run -m pytest`
- Generated report: `coverage report -m`

4.2. Coverage Results

Module	Statements	Missed	Coverage
Order_Placement.py	119	17	86%
Payment_Processing.py	67	3	96%
Restaurant_Browsing.py	52	4	92%
User_Registration.py	45	1	98%
TOTAL	336	26	**92%**

```
Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
Order_Placement.py              119     17    86%   34, 76-79, 96-97, 110-
114, 136, 185-186, 203, 235, 364
Payment_Processing.py            67      3    96%   39, 111, 187
Restaurant_Browsing.py           52      4    92%   137, 151-152, 204
User_Registration.py             45      1    98%   131
test_advanced_boundary.py        53      1    98%   67
-----------------------------------------------------------
TOTAL                           336     26    92%
```

PROF



4.3. Analysis of Untested Areas

- Untested lines are mostly error branches, rare input scenarios, or legacy code paths.
- New tests improved coverage, but some branches remain due to complexity or low risk.

5. Isolation Techniques: Stubs, Mocks, and Fakes

5.1. Definitions

- **Stub:** Simplified function returning fixed values.
- **Mock:** Simulated object recording interactions.
- **Fake:** Lightweight working implementation for integration.

5.2. Implementation Examples

Stub Example:

```
import unittest

def fetch_menu_stub():
    return [{"name": "Pizza", "price": 10.0}]

class TestMenuStub(unittest.TestCase):
    def test_menu_stub_returns_fixed_value(self):
        menu = fetch_menu_stub()
        self.assertEqual(menu, [{"name": "Pizza", "price": 10.0}])
```

Mock Example:

```
import unittest
from unittest.mock import Mock

class TestPaymentMock(unittest.TestCase):
    def test_payment_gateway_charge_called(self):
        mock_payment_gateway = Mock()
        mock_payment_gateway.charge.return_value = True
        result = mock_payment_gateway.charge(100)
        self.assertTrue(result)
        mock_payment_gateway.charge.assert_called_with(100)
```

PROF

Fake Example:

```
import unittest

class FakePaymentGateway:
    def charge(self, amount):
        return amount < 1000  # Accepts charges below 1000

class TestPaymentFake(unittest.TestCase):
    def test_fake_gateway_accepts_small_amount(self):
        gateway = FakePaymentGateway()
```

```
            self.assertTrue(gateway.charge(500))

        def test_fake_gateway_rejects_large_amount(self):
            gateway = FakePaymentGateway()
            self.assertFalse(gateway.charge(1500))
```

```
muditha@lap:~/SoftwareTesting$ python -m unittest test_isolation_examples.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
muditha@lap:~/SoftwareTesting$
```

---

6. Reflections & Discussion

- **Coverage Tool Impact:** Helped identify and close coverage gaps, focusing efforts on critical branches.
- **Challenges:** Integrating mocks for legacy code, designing meaningful fakes, and balancing test isolation vs realism.
- **Test Quantity vs Quality:** Quality improved by targeting edge cases and using isolation techniques, not just increasing test count.

---

7. Conclusion & Recommendations

- Advanced testing improved reliability and security.
- Recommend integrating automated security scanning and expanding integration/system tests.
- Maintain regular code reviews and update dependencies.
